COMP102: DATA STRUCTURES WITH PYTHON

Static and Dynamic Arrays

Rachad Bentbib
April 03, 2026

Plan

1 Static Arrays

Definition and Properties
Python Implementations

2 Dynamic Arrays

Definition and Properties
Python Implementations

3 Amortized Analysis

Aggregate Analysis
Accounting Method
Potential Method

Static Arrays

Definition and Properties

Definition of an Array

Definition 1.1 Array

An array of size n is a contiguous block of memory slots of n elements of identical type.

Definition of an Array

Definition 1.2 Array

An array of size n is a contiguous block of memory slots of n elements of identical type.

Properties:

- Fixed capacity

Definition of an Array

Definition 1.3 Array

An array of size n is a contiguous block of memory slots of n elements of identical type.

Properties:

- Fixed capacity
- Homogeneous elements

Definition of an Array

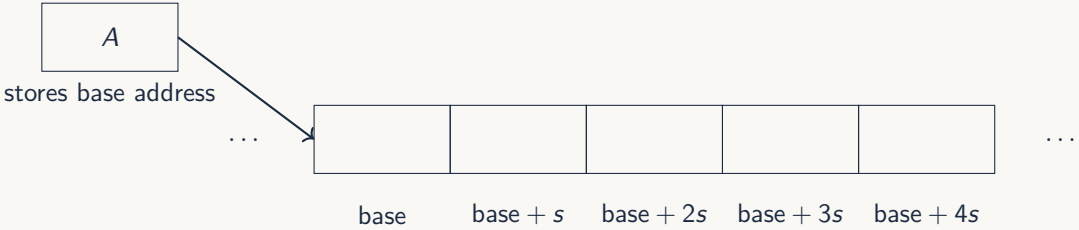
Definition 1.4 Array

An array of size n is a contiguous block of memory slots of n elements of identical type.

Properties:

- Fixed capacity
- Homogeneous elements
- Constant-time access to any element by its index

Array as a Memory Block



Since each element has size s bytes:

$$\text{address}(A[i]) = \text{base} + i \cdot s$$

Accessing Array Elements

Algorithm 1: Access Array Element

Input: Array A, index i
Output: Element $A[i]$

```
if i < 0 or i ≥ n then
    error
addr ← base(A) + i · s
value ← MEMORY[addr]
return value
```

Address Arithmetic

Proposition 1.1 Constant-Time Access

Accessing the ith element of an array takes $O(1)$ time and space.

Proof.

Address computation uses:

- One multiplication
- One addition
- One memory access

Each operation is $O(1)$ in time and space.

Copying the first k elements of an array

Procedure CopyArray(A, B, k)

Input: An array A of size n, an array B of size m, an integer k

```
// copy the k first elements of A into B
if k > n or k > m then // more elements to copy than the sizes of A and B
    error
for i ← 0 to k − 1 do // copy elements
    B[i] ← A[i]
```

Copying the first k elements of an array

Procedure CopyArray(A, B, k)

Input: An array A of size n, an array B of size m, an integer k

```
// copy the k first elements of A into B
if k > n or k > m then // more elements to copy than the sizes of A and B
    error
for i ← 0 to k − 1 do // copy elements
    B[i] ← A[i]
```

Takes $\Theta(k)$ time and $O(1)$ space.

Python Implementations

Static Arrays in Python?

A static array:

- has fixed capacity;
- stores elements contiguously;
- has homogeneous type.

Static Arrays in Python?

A static array:

- has fixed capacity;
- stores elements contiguously;
- has homogeneous type.

Python's built-in `list` is **not** a static array:

- it is dynamic;
- automatically resizes;

Static Arrays in Python?

A static array:

- has fixed capacity;
- stores elements contiguously;
- has homogeneous type.

Python's built-in `list` is **not** a static array:

- it is dynamic;
- automatically resizes;

For static arrays in Python, we can use:

- the `array` module
- the `ctypes` module (not in the scope of this course)

The array Module

Python illustration

```python
import array

A = array.array('i', [1, 2, 3, 4])  # 'i' is a type code for int
print(A[2])
```

Output:

```
3
```

Remark 1.1.

The type of the elements is specified by a type code. The module supports a limited number of types; see the documentation.

The ctypes module (not in the scope of this course)

ctypes provides C compatible data types, and allows calling functions in DLLs or shared libraries. It can be used to wrap these libraries in pure Python.

Python illustration

```python
import ctypes

FourIntArray = 4 * ctypes.c_int
A = FourIntArray(1, 2, 3, 4)
print(A[2])
```

Output:

```
3
```

Dynamic Arrays

Definition and Properties

Motivation

Static arrays have fixed capacity. In some cases, we don't know the number of elements to store in advance.
We must provide efficient resizing, both in time and space.

Dynamic Array

Definition 2.1

A *dynamic array* `A` is a data structure consisting of a static array `A.arr` of size `A.size`, called *capacity*, that stores a dynamic number `A.num` of elements of identical type.
The size and number of elements of a dynamic array changes in the course of a program as elements are *inserted* or *deleted*.

Dynamic Array

Definition 2.2

A *dynamic array* `A` is a data structure consisting of a static array `A.arr` of size `A.size`, called *capacity*, that stores a dynamic number `A.num` of elements of identical type.
The size and number of elements of a dynamic array changes in the course of a program as elements are *inserted* or *deleted*.

For $0 \leq i \leq n-1$, we let $A[i]$ be $(A.arr)[i]$.
One always has $0 \leq A.num \leq A.size$.

Dynamic Array

Definition 2.3

A *dynamic array* `A` is a data structure consisting of a static array `A.arr` of size `A.size`, called *capacity*, that stores a dynamic number `A.num` of elements of identical type.
The size and number of elements of a dynamic array changes in the course of a program as elements are *inserted* or *deleted*.

For $0 \leq i \leq n-1$, we let $A[i]$ be $(A.arr)[i]$.
One always has $0 \leq A.num \leq A.size$.

Assumption 2.3 Resizing strategy

- A dynamic array is initially empty: $A.arr = NIL$, $A.size = A.num = 0$.
- When adding an element and $A.num = A.size$: double capacity.
- When $A.num \leq A.size/4$: halve capacity.

Dynamic Array Expansion

Original array $A.arr$ $(A.size = A.num = 4)$

a_0	a_1	a_2	a_3

Dynamic Array Expansion

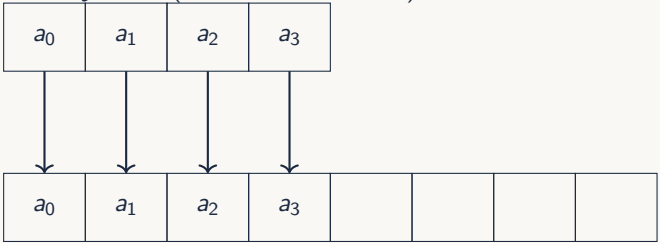
Original array $A.arr$ $(A.size = A.num = 4)$

a_0	a_1	a_2	a_3

New array B **of size 8**

Dynamic Array Expansion

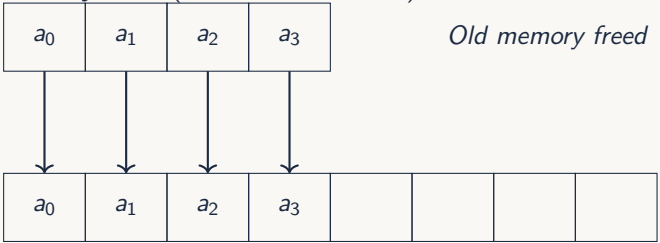
Original array $A.arr$ $(A.size = A.num = 4)$



New array B **of size 8**

Dynamic Array Expansion

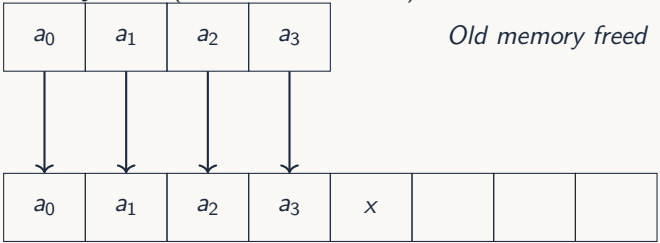
Original array $A.arr$ $(A.size = A.num = 4)$



New array B **of size 8**

Dynamic Array Expansion

Original array $A.arr$ $(A.size = A.num = 4)$



New array B **of size 8**

Operation: Insert at the end

Procedure InsertEnd(A, x)

Input: Dynamic array A, element x

```
// insert element x at the end of A
if A.size = 0 then // if empty, start with a single slot
    allocate A.arr with 1 slot
    A.size ← 1
if A.num = A.size then // if not enough space, expand
    allocate a new array B of size 2 × A.size
    CopyArray (A.arr, B, A.num)
    free A.arr // the memory occupied by A.arr becomes free
    A.arr ← B
    A.size ← 2 × A.size
A[A.num] ← x
A.num ← A.num + 1
```

Dynamic Array Contraction

Original array: $A.num = 3$, $A.size = 8$

a_0	a_1	a_2					

Dynamic Array Contraction

When deleting: $A.num = 2$, $A.size = 8 = 4 * A.num$

a_0	a_1	a_2					

Dynamic Array Contraction

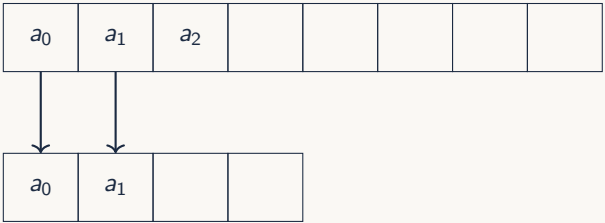
When deleting: $A.num = 2$, $A.size = 8 = 4 * A.num$

a_0	a_1	a_2					

New array B of size 4

Dynamic Array Contraction

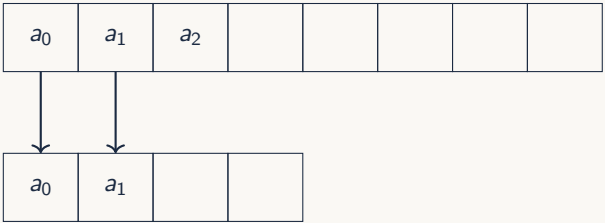
When deleting: $A.num = 2$, $A.size = 8 = 4 * A.num$



New array B of size 4

Dynamic Array Contraction

When deleting: $A.num = 2$, $A.size = 8 = 4 * A.num$



Old memory freed

New array B of size 4

Operation: Delete last element

Procedure DeleteLast(A)

Input: Dynamic array A

```
if A.num = 0 then // nothing to delete
    error
A.num ← A.num − 1 // note that it doesn't free the memory occupied by the last element itself
if A.num ≤ A.size/4 then // if too much free space, contract
    allocate a new array B of size 1/2 A.size
    CopyArray (A.arr, B, A.num)
    free A.arr
    A.arr ← B
    A.size ← ⌊1/2 A.size⌋
```

Python Implementations

The `dynamic_array` Module

Educational Implementation of Dynamic Arrays

The module provides a class implementing a **dynamic array**.

- Stores references to Python objects.
- Automatically grows and shrinks following the resizing policy described in this lecture.

Internal Representation

The class maintains three attributes:

- `_arr` : array storing elements
- `_capacity` : allocated memory size
- `_count` : number of elements stored

Conceptually:

$$\text{capacity} \geq \text{number of elements}$$

a_0	a_1	a_2	a_3		$\cdots$		

Only the first `_count` cells contain valid elements.

Public Interface

The main operations are:

- `DynamicArray(container=None)`
  - create an empty array or initialize from a container
- `insert_last(x)`
  - append an element
- `delete_last()`
  - remove and return last element
- `len(A)`
  - number of stored elements
- `A[i]`
  - access element by index
- `capacity()`
  - return the capacity of the array

The class also supports:

- iteration
- printing

CPython List Growth Formula

When a list must grow, CPython computes the new capacity using

```
new_allocated = ((size_t)newsize + (newsize >> 3) + 6) & ~(size_t)3
```

Explanation

- `newsize`
  number of elements that must fit in the list.
- `newsize >> 3`
  bit shift dividing by 8: $\left\lfloor \frac{\texttt{newsize}}{8} \right\rfloor$
  This adds about 12.5% extra capacity.
- `+ 6`
  small constant that avoids frequent reallocations for small lists.
- `& ~3`
  rounds the result to a multiple of 4 for memory alignment.

Compared with our doubling strategy, CPython grows lists more gradually: it uses less unused memory, but performs reallocations more often.

Amortized Analysis

Motivation for Amortized Analysis

Some operations have different costs depending on the state of the data structure.

Example: inserting into a dynamic array.

- Most insertions cost $\Theta(1)$.
- Occasionally, when the array is full, a resize occurs and all elements must be copied, with a cost of $\Theta(n)$.

Worst-case analysis alone does not reflect the typical cost of operations.

Aggregate Analysis

Aggregate Analysis

Definition 3.1 Aggregate Method

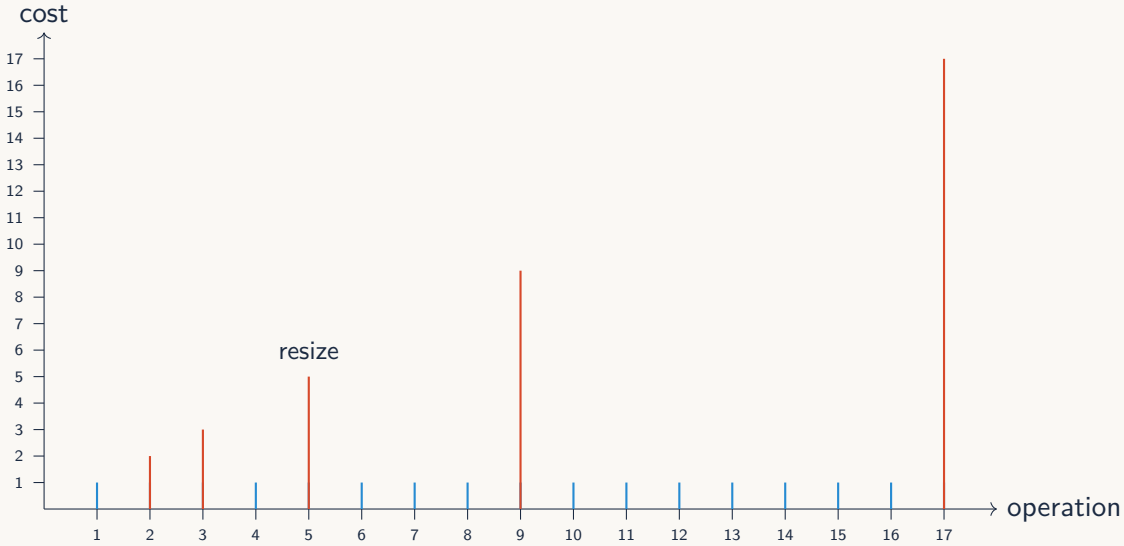
Let c_i be the cost of the i-th operation.
If we can prove that for any sequence of n operations, the total cost $\sum_{i=1}^{n} c_i$ is in the worst case $T(n)$, then the amortized cost $\hat{c}_i$ of each operation is the average $\frac{T(n)}{n}$.

Aggregate Analysis: Dynamic Array Insert

Assume we successively insert n element to an empty dynamic array.
The cost of the ith insert operation is

$$c_i = \begin{cases} i & \text{if } i-1 \text{ is a power of 2,} \\ 1 & \text{otherwise.} \end{cases}$$

Aggregate Analysis: Dynamic Array Insert



Total cost up to $n = 17$ is linear: spikes are rare and form a geometric series.

Aggregate Analysis: Dynamic Array Insert

$$
\begin{aligned}
\sum_{i=1}^{n} c_i &\leq n + \sum_{j=0}^{\lfloor \log(n) \rfloor} 2^j \\
&\leq n + \frac{2^{\lfloor \log(n) \rfloor)+1} - 1}{2-1} \\
&\leq n + 2n \\
&\leq 3n.
\end{aligned}
$$

Therefore

$$
\begin{aligned}
\hat{c}_i &\leq \frac{3n}{n} \\
&\leq 3 \\
&= O(1).
\end{aligned}
$$

Accounting Method

Accounting Method

Definition 3.2 Accounting Method

Each operation is assigned an amortized cost $\hat{c}_i$.
Extra cost is stored as **credit** that can be used to pay for future expensive operations.
The credits must never become negative.

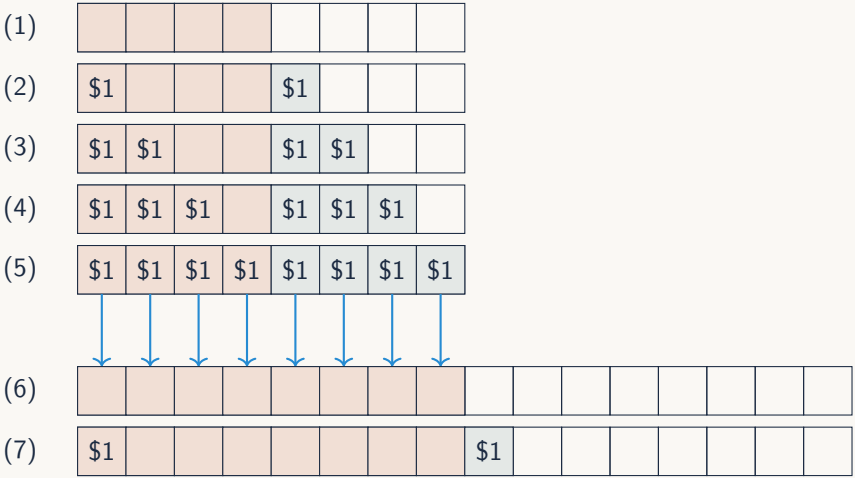
Accounting Method: Dynamic Array Insert

Assume we successively insert n element to an empty dynamic array.
Assign an amortized cost of 3 to each insertion.

- pay 1 for insertion
- store 1 credit with the inserted element, and 1 credit to an element already in the array with 0 credit

When resizing occurs, the stored credits pay for the copying of elements.

Accounting Method: Credit Distribution



Credits stored on elements pay for future resizing operations

Potential Method

Potential Method

Definition 3.3 Potential Method

Let D_0 be an initial data structure.
Let D_i be the data structure that results after applying the ith operation to D_{i-1}, with cost c_i.
A **potential function** Φ maps each data structure D_i to a real number $\Phi(D_i)$ such that for any i:

$$\Phi(D_i) \geq \Phi(D_0).$$

The amortized cost is

$$\begin{aligned} \hat{c}_i &:= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= c_i + \Delta\Phi_i \end{aligned}$$

Potential Method

The total amortized cost of i operations is:

$$\begin{aligned}\sum_{j=1}^{i} \hat{c}_j &= \sum_{j=1}^{i} (c_j + \Phi(D_j) - \Phi(D_{j-1})) \\ &= \sum_{j=1}^{i} c_j + \sum_{j=1}^{i} (\Phi(D_j) - \Phi(D_{j-1})) \\ &= \sum_{j=1}^{i} c_j + \Phi(D_i) - \Phi(D_0)\end{aligned}$$

Potential Method: Dynamic Array Insert

Assume we successively insert n element to an empty dynamic array. Let D_i be the array we obtain after inserting i elements.
Let $\Phi(D_i) := 2i - C(D_i)$, where $C(D)$ is the capacity of the dynamic array D. In particular, $\Phi(D_0) = 0 - 1 = -1$.
Since the array's capacity doubles whenever inserting into a full array is attempted, one can prove by induction that for $i \geq 1$: $\Phi(D_i) \geq 0 > -1 = \Phi(D_0)$.

- **Insertion without resize:** $c_i = 1$, $\Delta\Phi_i = (2i - C) - (2(i-1) - C) = 2$, thus $\hat{c}_i = 3$.
- **Insertion with resize:** $c_i = \underbrace{i-1}_{\text{copy}} + 1 = i$, and $C(D_{i-1}) = i - 1$, and $C(D_i) = 2C(D_{i-1})$, thus $\Phi(D_{i-1}) = 2(i-1) - C(D_{i-1})$ and $\Phi(D_i) = 2i - 2C(D_{i-1})$, hence $\Delta\Phi_i = 2 - C(D_{i-1}) = 3 - i$ and $\hat{c}_i = 3$.

We deduce that the amortized complexity of insertion is $O(1)$.